

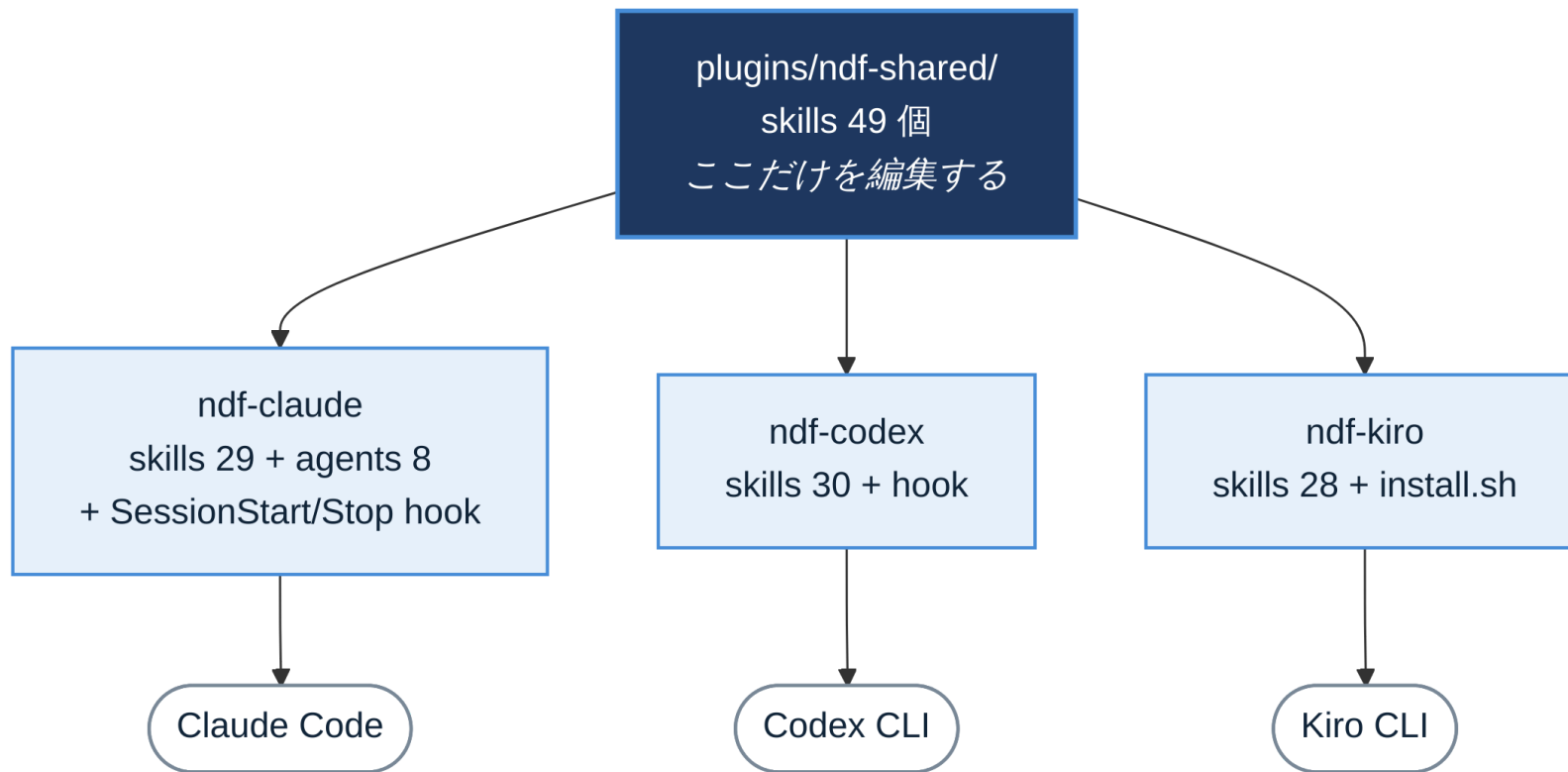
ai-plugins

NDF、実際のところ どう使うのか

社内勉強会 / 2026-08-06

Claude Code ・ Codex CLI ・ Kiro CLI 対応

まず全体像を1枚で



編集するのは `plugins/ndf-shared/` の **1か所だけ**。そこから3ランタイム分の配布物が生成されます。
ほかに MCP プラグインが **10個**（Serena / BigQuery / Playwright / Chrome DevTools / Redash など）。

インストールは2コマンド (Claude Code)

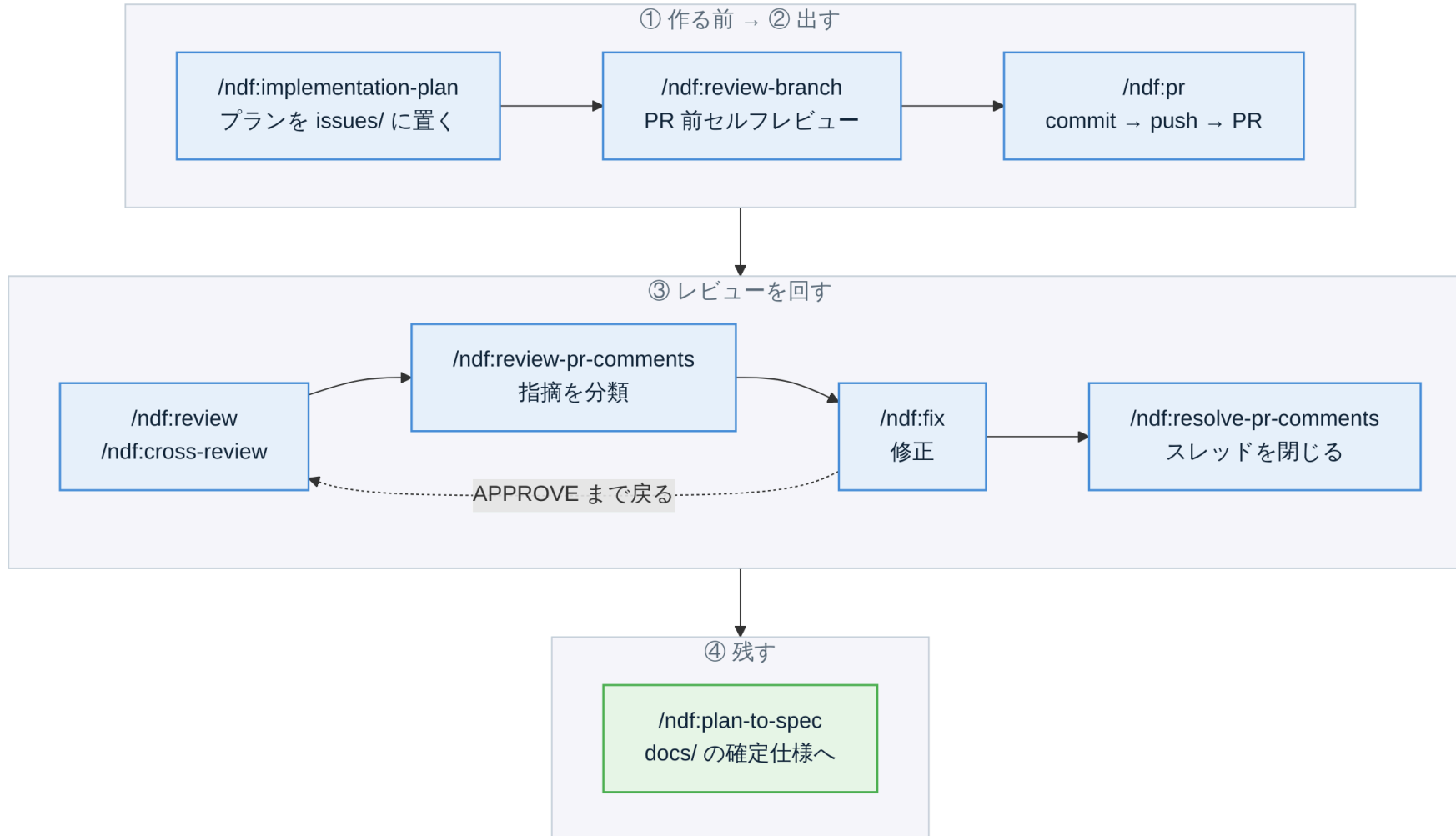
```
# 1. マーケットプレイスを登録（初回だけ）
/plugin marketplace add https://github.com/devbasex/ai-plugins

# 2. NDFを入れる
/plugin install ndf@ai-plugins
```

入れると使えるようになるもの:

スキル 29個	/ndf:pr , /ndf:review ... スラッシュで直接呼べる
エージェント 8個	director / corder / qa / debugger / devops-engineer など
フック 2種	SessionStart (transcript保持を90日に維持) / Stop (AI要約+Slack通知)

今日の主役: PRを1本出すまでの流れ



① 作る前に書く — `/ndf:implementation-plan`

実装を始めるとき、`issues/` にプランファイルがあるか確認して、なければ作る。

作るケース

複数ファイルにまたがる変更 / 新機能 / 既存ロジックの大幅変更 / DBマイグレーション

作らないケース

typo修正 / 設定値だけの変更 / 1ファイルで完結する軽微な修正

PR作成時にプランが無ければ、**会話履歴** + `git log` + `git diff` から自動生成してからPRを作ります。

ファイル名に日本語は使いません（Git / CI / 検索ツール互換性のため）。`issues/TASK-1234_add-export-api.md` のように。

② 出す — `/ndf:review-branch` → `/ndf:pr`

PR を出す前に自分で見る

```
/ndf:review-branch  
/ndf:review-branch security  
/ndf:review-branch performance  
/ndf:review-branch tests
```

mainとの差分を、品質・セキュリティ・パフォーマンス観点でセルフレビュー。

そのままPRにする

```
/ndf:pr  
/ndf:pr --draft  
/ndf:pr "エクスポートAPIを追加"  
/ndf:pr qa/staging
```

commit → push → PR作成 まで一括。
既にPRがあれば本文を最新差分で更新。

`/ndf:pr` はデフォルトブランチへの直接コミットを拒否します。うっかり main で作業してても止まります。

③ 出した後 — /ndf:review

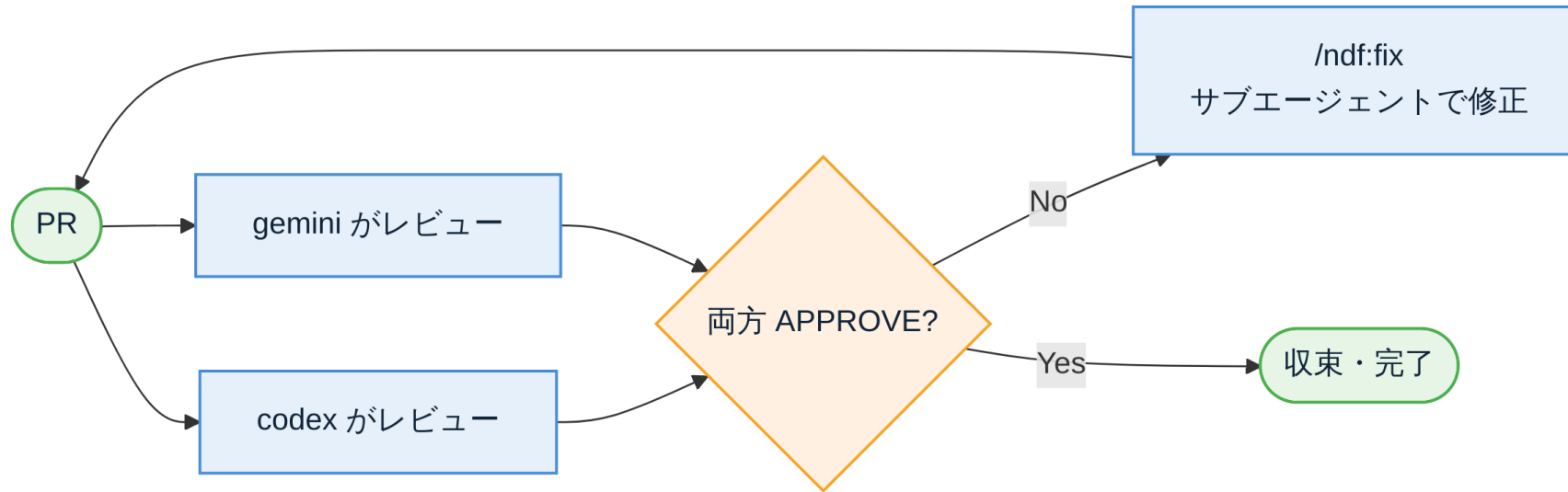
```
/ndf:review          # 直前のPRを Claude 自身がレビュー
/ndf:review 123      # PR番号を指定
/ndf:review 123 codex # Codex CLI に委譲
/ndf:review 123 gemini # Gemini CLI に委譲
```

第二引数でレビュアーを差し替えられるのがポイント。

	使いどころ
省略 (Claude)	ふだんのレビュー。文脈を持っているので速い
codex / gemini	第二意見がほしいとき 。自分が書いたコードを別のAIに見せる

/ndf:review-branch はローカル差分 (PR前)、 /ndf:review は GitHub 上の既存PR。対象が違うだけです。

④ 両AIが納得するまで回す — `/ndf:cross-review`



```
/ndf:cross-review 123
/ndf:cross-review 123 --focus "ドキュメントとコードの整合性を重点的に"
/ndf:cross-review 123 --max-rounds 4 --only codex
```

⑤ 指摘を捌く3段 — 分類 → 修正 → クローズ

コマンド	やること
<code>/ndf:review-pr-comments</code>	READ-ONLY 。全コメントを読んで対応可否と優先度を判定。修正は一切しない
<code>/ndf:fix</code>	分類結果をもとに実際に修正。 <code>--severity-min</code> / <code>--defer-nit</code> で範囲を絞れる
<code>/ndf:resolve-pr-comments</code>	対応済みコメントに返信し、スレッドを resolved にする

```
/ndf:review-pr-comments 123      # まず見るだけ
/ndf:fix 123 --defer-nit         # nit は後回しにして直す
/ndf:resolve-pr-comments 123     # 返信して閉じる
```

「読む」と「直す」を分けているのは、**全部直すのが正解とは限らない**から。先に分類だけさせると判断できます。

⑥ 書いたものを資産に残す

/ndf:plan-to-spec

実装が終わった `issues/` のプランを、
現在のコードと一致する確定仕様書に
書き直して `docs/` へ移す。

消すもの:

開発中の履歴 / TODO / PR分割 /
作業チェックリスト / 未採用案

関連: `/ndf:investigation-rules` (「～が無い」と書くなら実行結果を添えろ)、`/ndf:problem-solving` (つじつま合わせをせず上流で直す)

/ndf:markdown-writing

読み手は**会話も検討過程も知らない第三者**、
という前提で書かせるルール。

- 説明文にテーブル名・カラム名などの内部識別子を持ち込まない
- 「案A」「以前は～だった」を残さない
- 否定的な結論にはエビデンス必須

Codex / Kiro でも同じスキルが使えます

Codex CLI

```
codex plugin marketplace add \  
  https://github.com/devbasex/ai-plugins  
codex plugin add ndf@ai-plugins
```

セッション内では `ndf:` 接頭辞で
30個のスキルが使えます。

Claude版との差: エージェント8個と SessionStart/Stopフックは無し。代わりに Playwright 系スキル5個が入ります。

Kiro CLI

```
git clone \  
  https://github.com/devbasex/ai-plugins.git  
bash plugins/ndf-kiro/install.sh  
kiro-cli chat
```

`.kiro/skills/` に**28個**、
`.kiro/agents/default.json` を生成。

`--with-slack` でStop時のSlack通知、`--with-codex` で
Codex CLI連携を追加。何度実行しても安全（冪等）。

まとめ: 今日から試す3ステップ

1. 入れる — `/plugin marketplace add ...` → `/plugin install ndf@ai-plugins`
2. 次のPRで `/ndf:pr` を叩く — commit・push・PR作成・本文更新まで任せてみる
3. レビューが付いたら `/ndf:review-pr-comments` — まず分類だけさせて、判断は自分でやる

スキルは覚えるものではなく、**繋がっている流れを1回なぞってみる**ものです。
プラン → PR → レビュー → 仕様書、を1本だけ通してみてください。

リポジトリ: <https://github.com/devbasex/ai-plugins> / 詳細は `docs/ndf-plugin-reference.md`